CSE 321 Study Question Examples

1. Sort the following functions from slowest to fastest in terms of their growth rates.
   a) $\log(n!), n^{\log n}, n^{1.54}$
   b) $2^{\log n}, 4^{\log n}, n^{2/3}$
   c) $7^{n/2}, \sqrt{n}, \ 8^{logn}$
   d) $2^n + n^3, n^n + 1000n, 3^n + n^3$

2. Calculate the running time of the codes below. Express your answer using the O notation.

Algorithm 1:

```
for i = 2 * n; i ≥ 1; i = i − 1 do
   for j = 1; j ≤ 1; j = j + 1 do
      for k = 1; k ≥ j; k = k * 3 do
         print(hello)
      end for
   end for
end for
```

Algorithm 2:

Assume the function *pow*(*k, x*) calculates x^k.

```
total = 0;
for k = 5; k ≤ n ; k = pow(k, x) do
   total = total + k
end for
```

3. Prove or Disprove.
   (a) $3^n = O(2^n)$
   (b) $n^2 + n = o(n^2)$
   (c) $4n^3 - 3n + 1 = \theta(n^2)$

4. Use the master method to give tight asymptotic bounds for the following recurrences.

   (a) $T(n) = 4T(n/2) + n$
   (b) $T(n) = 4T(n/2) + n^2$
   (c) $T(n) = 4T(n/2) + n^2 logn$
   (d) $T(n) = 4T(n/2) + n^3$

5. Solve the following recurrences using substitution.
   (a) $T(n) = 2T(n/2) + n/\log(n), T(1) = 1$
   (b) $T(n) = 2T\sqrt{n} + 1, T(1) = 1$
   (c) $T(n) = T(n/2) + 1, \ T(1) = 1$

(d) T(n) = 8T(n/2) + n^2 , T(1) = 1

6. Find the time complexity of the following recurrence relations by using the characteristicequation method.
   (a) $T(n) = 5T(n-1) - 6T(n-2) + 4n - 3$, T(0) = 1, T(1) = 2
   (b) $T(n) = 5T(n-1) - 6T(n-2) + 2^n$, T(0) = 1, T(1)=2
   (c) $T(n) = 4T(n-1) - 4T(n-2)$, T(0) = 1, T(1)=2

7. Write a recurrence relation for the following functions and analyze their time complexity T(n). Find the particular solution for the related recurrence relation.

Algorithm 3:

```
func (int n, long count)
int i;
if n < 2 then
    count = n;
    return;
end if
for i = 1; i ≤ n; i++ do
    count++;
end for
func(n − 2, count)
```

Algorithm 4:

```
int foo(int n)
s = 1;
for i = 0; i < n; i++ do
    s = s + foo(i);
end for
return s;
```

8. Describe and analyze a logarithmic time algorithm that finds the number of 1's given a sorted binary array. Obtain the worst-case, best-case and average-case complexities.

9. Describe and analyze a θ(n) time algorithm that searches two numbers in a given sorted array whose sum is exactly x.